

AI INTERNSHIP

Task 1

Breast Cancer Classification

Machine Learning model development, evaluation and validation

Candidate	C. Rajalakshmi
Platform	Google Colab
Dataset	Breast Cancer Wisconsin Dataset
Models	Logistic Regression • Random Forest

Submission-ready project report containing the implementation overview, evaluation results, cross-validation, visual outputs, reproducibility notes, and repository reference.

01 Executive Summary

This project implements a binary classification workflow using the Breast Cancer Wisconsin dataset available through scikit-learn. Two supervised learning models were trained and evaluated: Logistic Regression and Random Forest.

The workflow includes dataset preparation, an 80/20 stratified train-test split, model training, multiple evaluation metrics, 5-fold cross-validation, and confusion matrix visualization.

569	30	2	5-fold
Samples	Features	Models	CV validation

02 Project Objective

To develop and compare machine learning classification models capable of classifying observations in the Breast Cancer Wisconsin dataset, while documenting their performance using standard classification metrics.

03 Dataset & Preparation

The dataset contains 569 observations and 30 numerical features. The data was split into 455 training samples and 114 testing samples using a stratified 80/20 split with a fixed random state of 42.

04 Technology Stack

Language: Python **Environment:** Google Colab

Libraries: Pandas, NumPy, Matplotlib, Scikit-learn

05 Methodology

Step 1 — Data Loading: Load the Breast Cancer Wisconsin dataset using scikit-learn and construct feature and target data structures.

Step 2 — Train/Test Split: Create an 80/20 stratified split to preserve the class distribution between training and testing data.

Step 3 — Model Training: Train Logistic Regression with StandardScaler inside a pipeline, and train a Random Forest Classifier with 100 estimators.

Step 4 — Evaluation: Calculate Accuracy, Precision, Recall, F1-Score and ROC-AUC.

Step 5 — Validation: Use 5-fold cross-validation to measure the mean accuracy and standard deviation across folds.

Step 6 — Visualization: Generate confusion matrices for both trained models.

Model Configuration

Model	Configuration
Logistic Regression	StandardScaler + LogisticRegression (max_iter=2000)
Random Forest	RandomForestClassifier (n_estimators=100, random_state=42)

06 Model Evaluation

The following values are taken directly from the completed notebook evaluation.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	98.25%	98.61%	98.61%	98.61%	99.54%
Random Forest	95.61%	95.89%	97.22%	96.55%	99.37%

5-Fold Cross-Validation

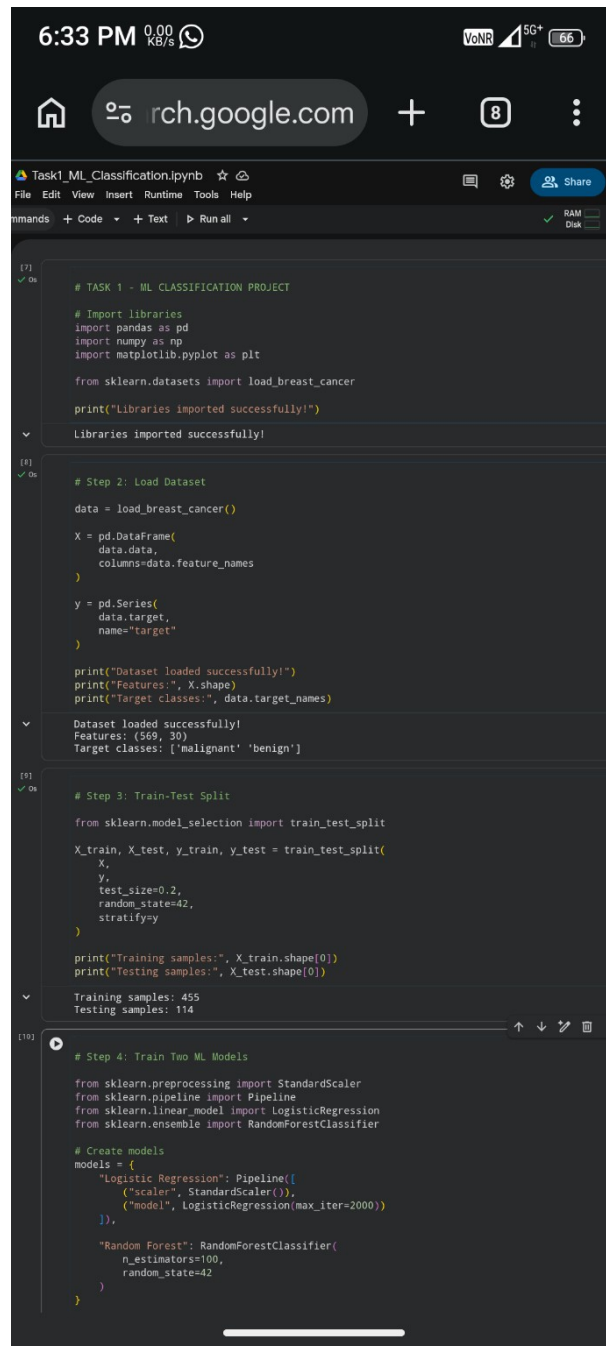
Model	Mean CV Accuracy	Standard Deviation
Logistic Regression	98.07%	0.006539
Random Forest	95.61%	0.022839

Notebook observation: Logistic Regression recorded the higher test accuracy and higher mean 5-fold cross-validation accuracy in this experiment.

07 Notebook Evidence

Selected screenshots from the executed Google Colab notebook are included below to document the implementation and generated outputs.

Dataset loading, train/test split and model training



The screenshot displays a Google Colab notebook interface on a mobile device. The top status bar shows the time as 6:33 PM, a data speed of 0.00 KB/s, VoNR 5G+ connectivity, and a 66% battery level. The browser address bar shows 'rch.google.com'. The notebook title is 'Task1_ML_Classification.ipynb'. The interface includes a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a toolbar with icons for commands, code, text, and running all cells. The notebook content is divided into three visible code cells, each with a green checkmark indicating successful execution.

```
[7] ✓ On
# TASK 1 - ML CLASSIFICATION PROJECT

# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer

print("Libraries imported successfully!")

Libraries imported successfully!

[8] ✓ On
# Step 2: Load Dataset

data = load_breast_cancer()

X = pd.DataFrame(
    data.data,
    columns=data.feature_names
)

y = pd.Series(
    data.target,
    name="target"
)

print("Dataset loaded successfully!")
print("Features:", X.shape)
print("Target classes:", data.target_names)

Dataset loaded successfully!
Features: (569, 30)
Target classes: ['malignant' 'benign']

[9] ✓ On
# Step 3: Train-Test Split

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])

Training samples: 455
Testing samples: 114

[10] ⌚
# Step 4: Train Two ML Models

from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

# Create models
models = {
    "Logistic Regression": Pipeline([
        ("scaler", StandardScaler()),
        ("model", LogisticRegression(max_iter=2000))
    ]),

    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        random_state=42
    )
}
```

Evaluation metrics and cross-validation



The screenshot shows a Jupyter Notebook titled 'Task1_ML_Classification.ipynb' on a mobile device. The interface includes a top status bar with the time '6:33 PM', network status 'VoNR', and battery level '66%'. Below the status bar is a browser-like address bar showing 'rch.google.com'. The notebook's toolbar includes options for 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. The code is organized into cells, with the first cell showing the training of two models: Logistic Regression and Random Forest. The second cell displays the evaluation results for these models, including accuracy, precision, recall, F1 score, and ROC-AUC. The third cell shows the import of the cross-validation module.

```
[10]: # Step 4: Train Two ML Models

from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

# Create models
models = {
    "Logistic Regression": Pipeline([
        ("scaler", StandardScaler()),
        ("model", LogisticRegression(max_iter=2000))
    ]),

    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        random_state=42
    )
}

# Train both models
for name, model in models.items():
    model.fit(X_train, y_train)
    print(name, "trained successfully!")

print("Both models trained successfully!")

... Logistic Regression trained successfully!
Random Forest trained successfully!
Both models trained successfully!

[11]: # Step 5: Model Evaluation

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report
)

results = []

for name, model in models.items():
    predictions = model.predict(X_test)
    probabilities = model.predict_proba(X_test)[:, 1]

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, predictions),
        "Precision": precision_score(y_test, predictions),
        "Recall": recall_score(y_test, predictions),
        "F1 Score": f1_score(y_test, predictions),
        "ROC-AUC": roc_auc_score(y_test, probabilities)
    })

results_df = pd.DataFrame(results)

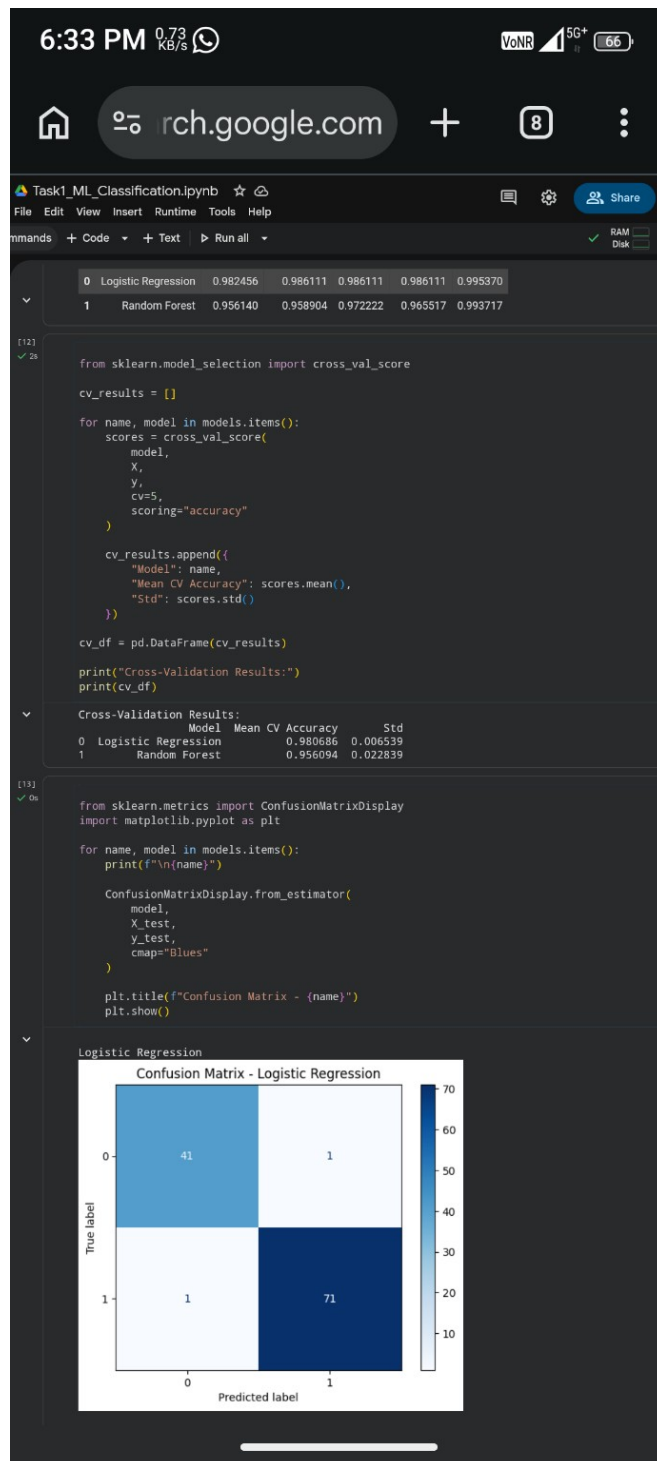
print("Model Evaluation Results:")
results_df

Model Evaluation Results:
   Model Accuracy Precision Recall F1 Score ROC-AUC
0 Logistic Regression  0.982456  0.986111  0.986111  0.986111  0.995370
1 Random Forest      0.956140  0.958904  0.972222  0.965517  0.993717

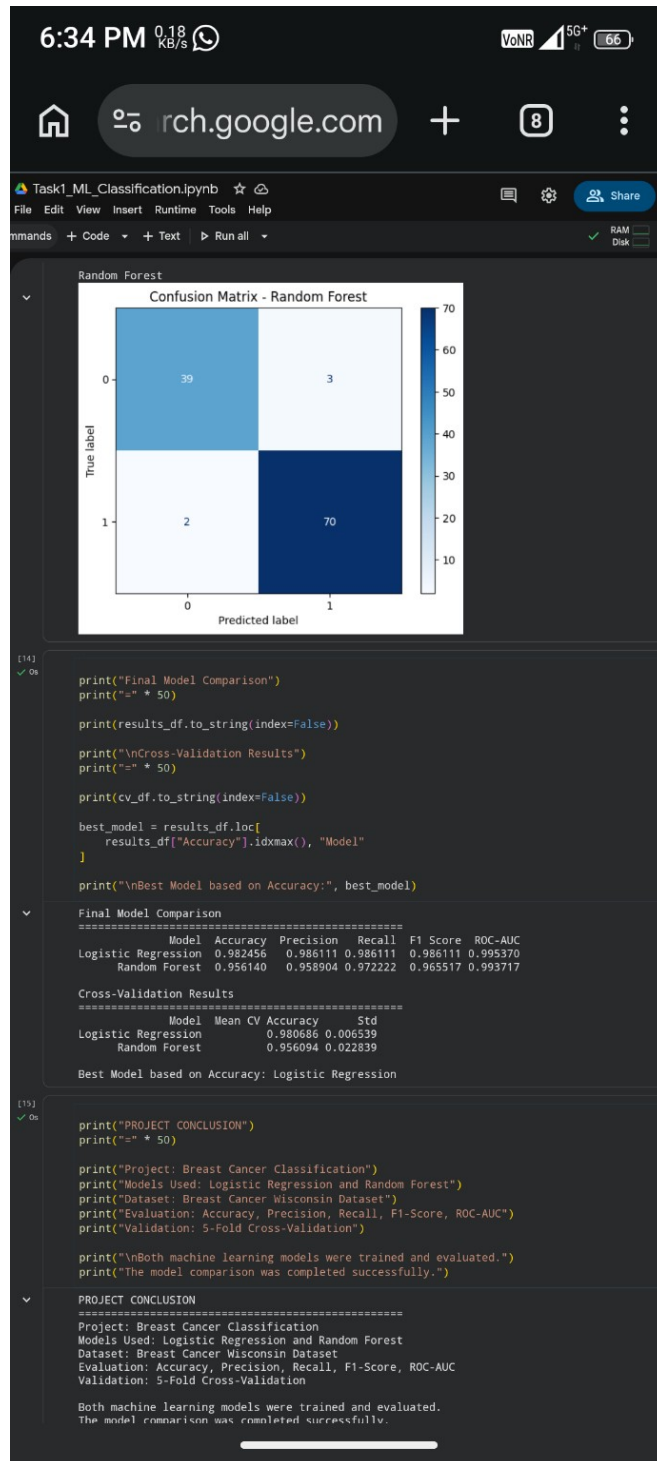
[12]: from sklearn.model_selection import cross_val_score

cv_results = []
```

Confusion Matrix — Logistic Regression



Confusion Matrix — Random Forest



08 Reproducibility

The notebook can be reproduced in Google Colab or a compatible Python environment.

Item	Details
Python environment	Python 3.x / Google Colab
Required libraries	pandas, numpy, scikit-learn, matplotlib
Install command	pip install pandas numpy scikit-learn matplotlib
Execution	Open the .ipynb file and run cells from top to bottom

09 Project Repository

GitHub Repository: github.com/crajalakshmi-2226/AlfidoTech

Notebook file: **Task1_Breast_Cancer_Classification.ipynb**

10 Conclusion

The machine learning classification workflow was successfully implemented using Logistic Regression and Random Forest. The models were trained on the Breast Cancer Wisconsin dataset and evaluated using multiple classification metrics, 5-fold cross-validation, and confusion matrices. The complete notebook and supporting documentation are maintained in the AlfidoTech GitHub repository.

Prepared for internship task submission